

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

Тема: UI-тестирование

Студенты гр. 4344,
4341

Кочененко А.А.
Благодарная К.О.
Палаев И.С.
Стиврин А.Д.

Руководитель

А.М. Шевелёва

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: Палаев И.С.

Группа: 4344

Тема практики: UI-тестирование сайта Demoblaze

Задание на практику:

Разработка и внедрение программного решения для автоматизированного UI-тестирования отдельных функциональных модулей веб-приложения с применением современного набора инструментов. В рамках практики необходимо сформировать детальный чек-лист, состоящий из 10 тестовых сценариев с суммарным количеством не менее 50 пользовательских действий, а также выполнить их программную реализацию на языке Java с использованием Selenide и JUnit 5. Архитектура проекта должна быть организована на основе паттерна Page Object Model и компонентного подхода, что позволяет разделить логику тестовых сценариев, описание страниц и работу с элементами интерфейса, упрощая поддержку и развитие кода. Также работа предполагает настройку системы логирования Log4j2 для отслеживания выполнения тестов, ведение совместной разработки через репозиторий GitHub и подготовку итогового отчета с описанием структуры классов, методов и соответствующих UML-диаграмм.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 18.07.2026

Дата защиты отчета: 18.07.2026

Студенты

Руководитель

Кочененко А.А.
Благодарная К.О.
Палаев И.С.
Стиврин А.Д.

А.М. Шевелёва

АННОТАЦИЯ

В отчёте рассматривается разработка и реализация автоматизированного UI-тестирования для демонстрационного интернет-магазина Demoblaze. В процессе работы были изучены основные возможности сайта и подготовлен чек-лист пользовательских сценариев, отражающих ключевые действия покупателя: переход по категориям товаров, открытие карточки продукта, добавление товаров в корзину, удаление позиций, проверка состояния корзины, оформление заказа и отмена оформления.

Для реализации тестов был использован язык программирования Java. Структура проекта построена с применением паттерна Page Object Model, благодаря чему тестовые сценарии отделены от логики взаимодействия с элементами веб-интерфейса. Такой подход позволил сделать код более понятным, упорядоченным и удобным для дальнейшего сопровождения.

Автоматизированные проверки выполнены с использованием фреймворка Selenide, библиотеки JUnit 5, системы сборки Maven и средств логирования. В рамках проекта были описаны классы страниц, элементы интерфейса и тестовые классы, а также подготовлена сопроводительная документация с описанием реализованных тестов, методов и UML-диаграмм. Итоговая версия проекта размещена в репозитории GitHub, где отражена история изменений и вклад участников команды.

SUMMARY

This report focuses on the development and implementation of automated UI testing for the demonstration online store Demoblaze. During the work, the main features of the website were studied, and a checklist of user scenarios was prepared. These scenarios reflect the key actions of a customer: navigating product categories, opening a product card, adding products to the cart, removing items, checking the cart state, placing an order, and canceling the checkout process.

The tests were implemented using the Java programming language. The project structure is based on the Page Object Model pattern, which separates test scenarios from the logic of interacting with web interface elements. This approach made the code more understandable, organized, and convenient for further maintenance.

The automated checks were performed using the Selenide framework, the JUnit 5 library, the Maven build system, and logging tools. As part of the project, page classes, interface element classes, and test classes were described. Supporting documentation was also prepared, including descriptions of the implemented tests, methods, and UML class diagrams. The final version of the project was published in a GitHub repository, where the change history and the contribution of each team member are reflected.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 ПОСТАНОВКА ЗАДАЧИ	9
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	27
2.1. Описание программных классов и их методов	27
2.2. UML-диаграмма классов	30
2.3. Реализация взаимодействия компонентов.....	31
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	32
4 ОТЗЫВ РУКОВОДИТЕЛЯ	34
ЗАКЛЮЧЕНИЕ	35
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	37

ВВЕДЕНИЕ

Предметной областью данной практики является автоматизированное UI-тестирование пользовательских сценариев веб-приложения. В качестве тестируемой системы был выбран демонстрационный интернет-магазин Demoblaze [4], позволяющий воспроизвести основные действия пользователя при покупке товара: переход по категориям, просмотр карточки продукта, добавление товаров в корзину, удаление товаров, оформление заказа и отмену оформления. Автоматизация таких сценариев позволяет сократить объем ручной проверки, снизить вероятность ошибок и убедиться в корректной работе ключевых элементов интерфейса при повторном запуске тестов.

Целью практики является проектирование и программная реализация набора автоматизированных UI-тестов для проверки основного пользовательского пути в интернет-магазине Demoblaze. Основной упор делается на построение понятной и поддерживаемой структуры проекта с использованием объектно-ориентированного подхода и паттерна Page Object Model, при котором тестовые классы отделены от классов страниц и элементов интерфейса. В проекте реализованы отдельные классы страниц MainPage, ProductPage и CartPage, отвечающие за взаимодействие с главной страницей, карточкой товара и корзиной соответственно.

Для достижения поставленной цели необходимо решить следующие задачи:

Провести анализ функциональных возможностей сайта Demoblaze и сформировать чек-лист тестовых сценариев, покрывающих основные действия пользователя при выборе товара, работе с корзиной и оформлении заказа.

Разработать архитектуру проекта на языке Java с применением паттерна Page Object Model [1] и компонентного подхода, выделив базовые классы страниц и элементов интерфейса. В проекте для этого используются классы BasePage, BaseElement, Button и Input, позволяющие отдельно описывать страницы, кнопки и поля ввода.

Реализовать автоматизированные тесты с использованием Selenide [2] и JUnit 5 [3], обеспечив проверку перехода по категориям, открытия карточки товара, добавления и удаления товаров из корзины, отображения пустой корзины, открытия формы заказа, заполнения формы корректными данными и успешного оформления покупки.

Вынести повторяющиеся тестовые данные, такие как названия категорий, товаров и цена товара, в отдельный класс Constants, чтобы упростить поддержку тестов и избежать дублирования строковых значений в коде.

Настроить базовое логирование выполнения тестов и действий с элементами интерфейса с помощью класса LoggerUtil, а также подготовить итоговую документацию с описанием реализованных тестов, классов, методов и UML-диаграмм проекта.

Вклад каждого участника:

Палаев И.С.: Архитектура и базовая настройка проекта: настройка pom.xml, подключение необходимых зависимостей, создание базовых классов для работы с элементами интерфейса BaseElement, Button, Input. Реализация класса BasePage с общими методами для страниц, а также класса LoggerUtil для логирования выполнения тестов и действий с элементами. Приведение работы с кнопками и полями ввода к единому виду, чтобы одни и те же действия можно было использовать в разных тестах и страницах.

Стиврин А.Д.: Реализация Page Objects для сайта Demoblaze: создание классов MainPage, ProductPage и CartPage, отвечающих за работу с главной страницей, карточкой товара и корзиной. Добавление методов для перехода по категориям, открытия товара, перехода в корзину, добавления товара, проверки данных на странице товара, удаления товара из корзины, открытия формы заказа и заполнения полей оформления покупки. Вынес основные тестовые данные, такие как названия товаров, категории и цена, в класс Constants, что упростило использование одинаковых значений в разных тестах.

Благодарная К.О.: создание чек-листа UI-тестирования для сайта Demoblaze, настройка базового тестового класса BaseTest, отвечающего за запуск браузера, открытие главной страницы сайта, установку размера окна, таймаута ожидания и закрытие браузера после выполнения теста. Тесты №1–4: проверка перехода по категориям Phones и Laptops, открытие карточки товара Samsung Galaxy S6, переход в корзину через верхнее меню, проверка добавления товара в корзину, включая появление сообщения Product added и отображение добавленного товара в таблице корзины.

Кочененко А.А.: Создание чек-листа UI-тестирования для сайта Demoblaze. Реализация тестов №5–9: добавление нескольких товаров в корзину, удаление товара из корзины, открытие формы оформления заказа, оформление заказа с корректными данными, отмена оформления заказа, проверка пустой корзины и появление товара после добавления в пустую корзину. Добавление assert-проверок для подтверждения наличия и отсутствия товаров в корзине, отображения формы заказа и сообщения об успешной покупке. Проверка соответствия реализованных тестов подготовленному чек-листу.

1 ПОСТАНОВКА ЗАДАЧИ

В итоговом проекте имеются 10 тестов:

1. Проверка перехода по категориям товаров Phones и Laptops.
2. Проверка открытия страницы товара Samsung Galaxy S6.
3. Проверка перехода в корзину через верхнее меню.
4. Проверка добавления товара в корзину.
5. Проверка добавления нескольких товаров в корзину.
6. Проверка удаления товара из корзины.
7. Проверка оформления заказа.
8. Проверка отмены оформления заказа.
9. Проверка отображения пустой корзины и добавления товара в пустую корзину.

Тест №1: Проверка перехода по категориям товаров

Входные данные:

Категория: Phones. Товар: Samsung Galaxy S6.

Категория: Laptops. Товар: Sony vaio i5.

Действия:

1. Открыть главную страницу сайта Demoblaze.
2. Нажать категорию Phones.
3. Убедиться, что открыт список товаров категории Phones и в списке отображается товар Samsung Galaxy S6.
4. Вернуться на главную страницу сайта.
5. Нажать категорию Laptops.
6. Убедиться, что открыт список товаров категории Laptops и в списке отображается товар Sony vaio i5.

В результате выполнения теста проверяется корректность перехода по категориям товаров на сайте Demoblaze. После выбора категории Phones пользователю отображается список телефонов, содержащий товар Samsung

Galaxy S6. После выбора категории Laptops отображается список ноутбуков, содержащий товар Sony vaio i5.

Результаты действий представлены на рисунках (см. рисунки 1 – 3).

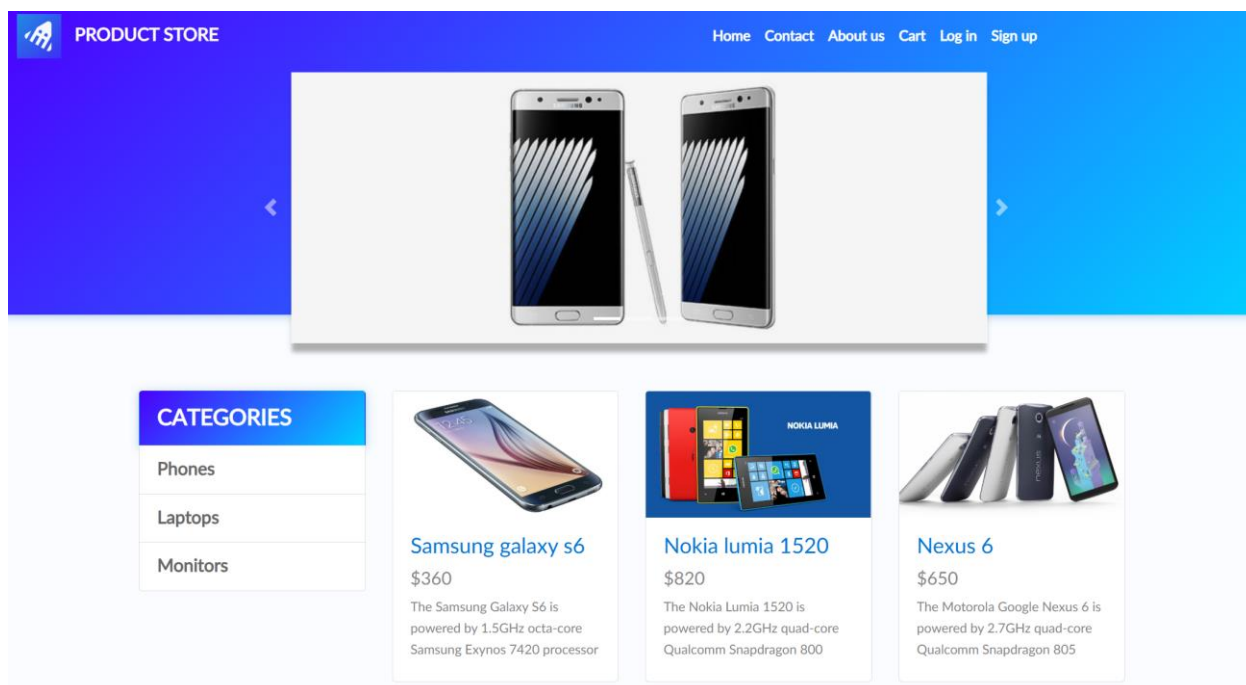


Рисунок 1 - Главная страница сайта Demoblaze

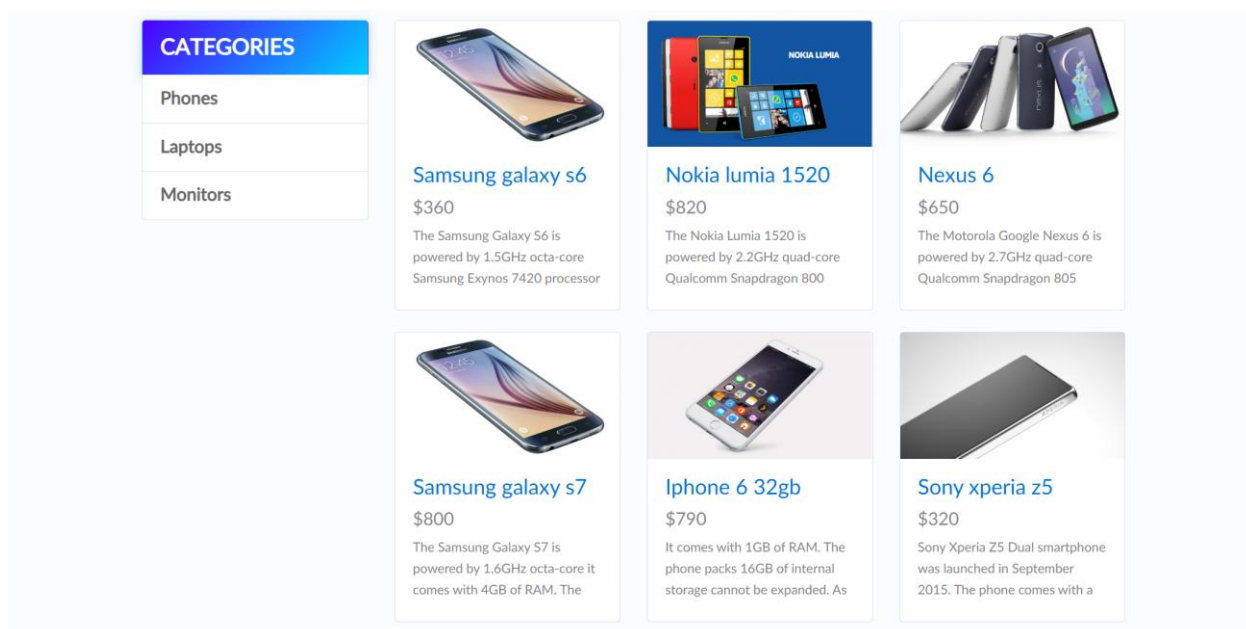


Рисунок 2 - Отображение товаров категории Phones

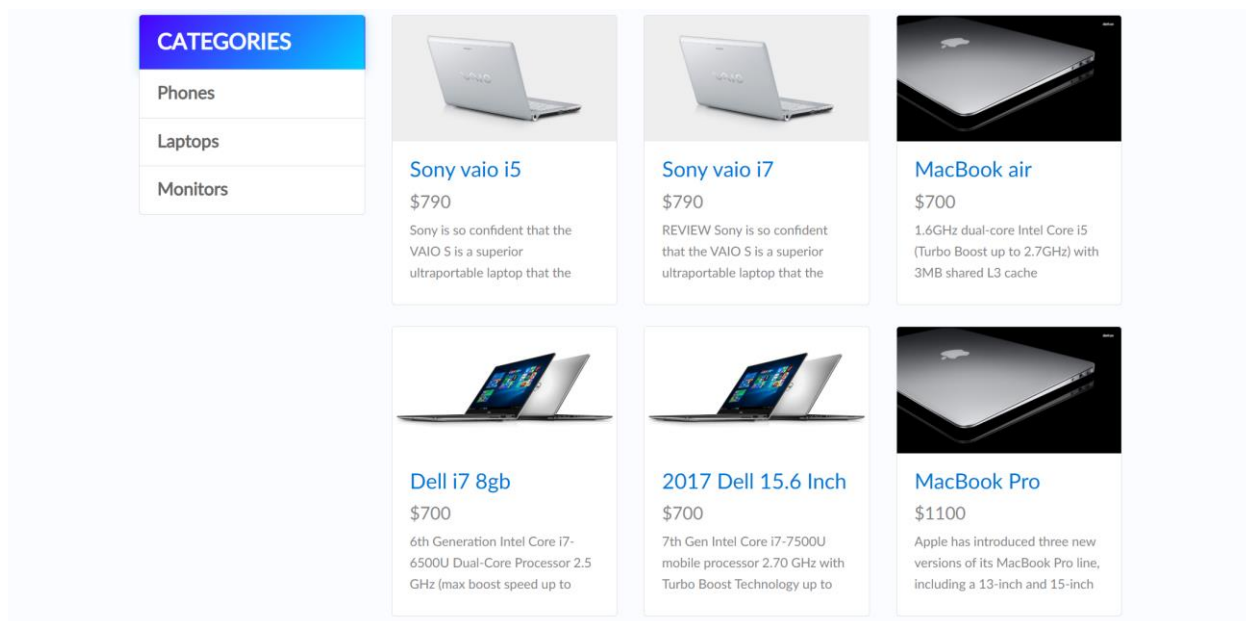


Рисунок 3 - Отображение товаров категории Laptops

Тест №2: Проверка открытия страницы товара

Входные данные:

Категория: Phones

Товар: Samsung Galaxy S6

Действия:

1. Открыть главную страницу сайта Demoblaze.
2. Перейти в категорию Phones.
3. Нажать на товар Samsung Galaxy S6.

В результате выполнения теста открывается карточка товара Samsung Galaxy S6. На странице товара отображаются название продукта, цена \$360, описание товара и кнопка Add to cart. Данный тест позволяет проверить корректность перехода из списка товаров в карточку выбранного продукта.

Результаты действий представлены на рисунках (см. рисунки 4 – 6).

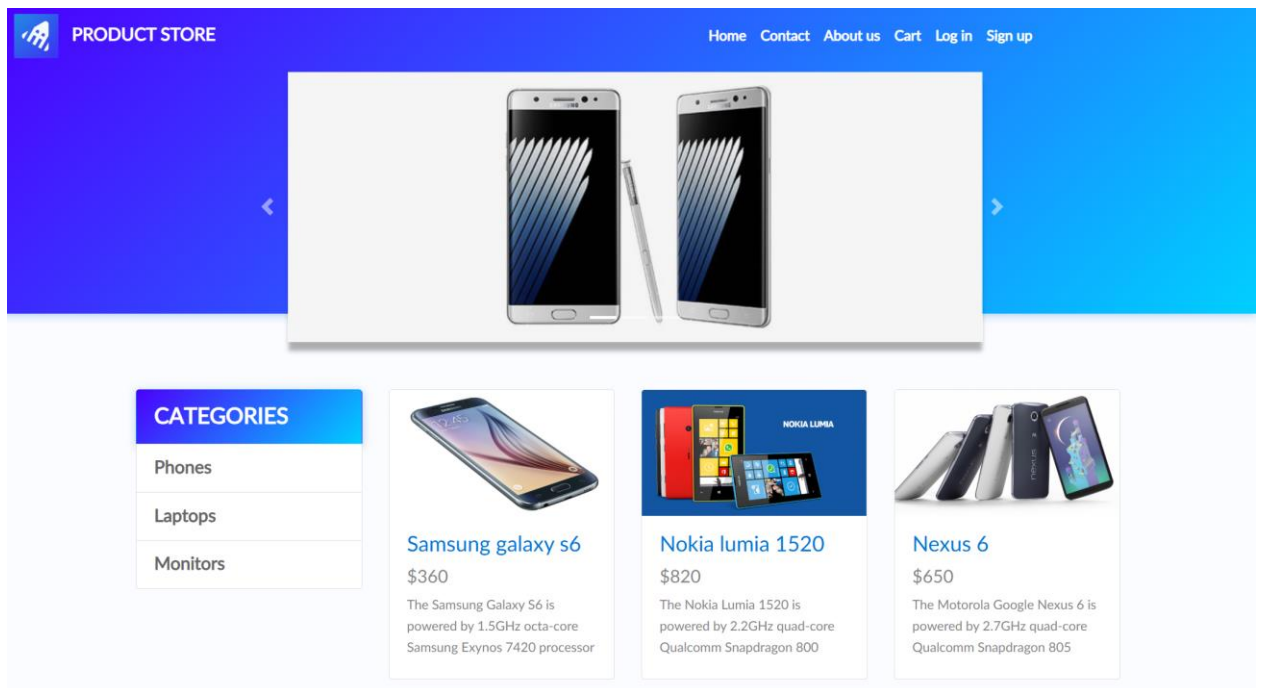


Рисунок 4 – Главная страница сайта

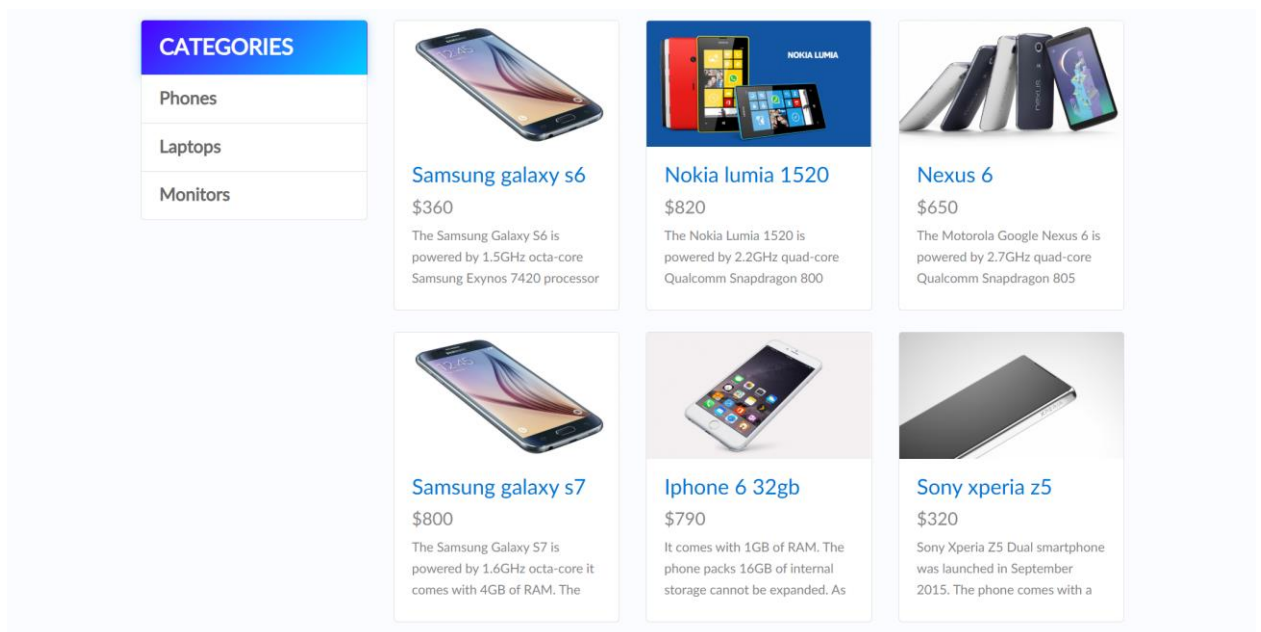


Рисунок 5 - Список товаров категории Phones

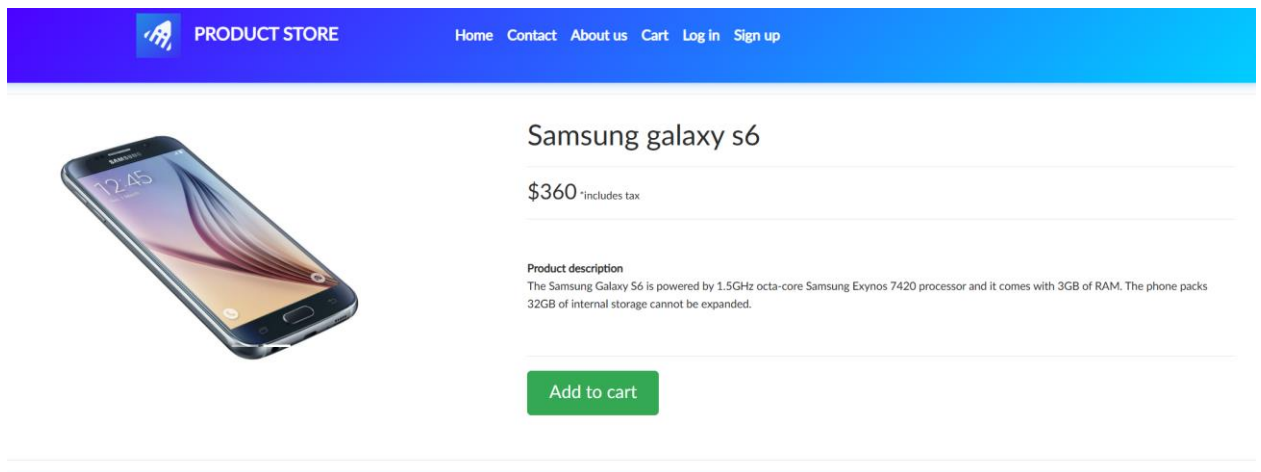


Рисунок 6 - Карточка товара Samsung Galaxy S6

Тест №3: Проверка перехода в корзину через верхнее меню

Входные данные: Пункт меню Cart

Действия:

1. Открыть главную страницу сайта Demoblaze.
2. Нажать пункт меню Cart.

В результате выполнения теста открывается страница корзины. На странице отображается таблица корзины с колонками Pic, Title, Price, х, а также кнопка Place Order.

Результаты действий представлены на рисунках (см. рисунок 7).

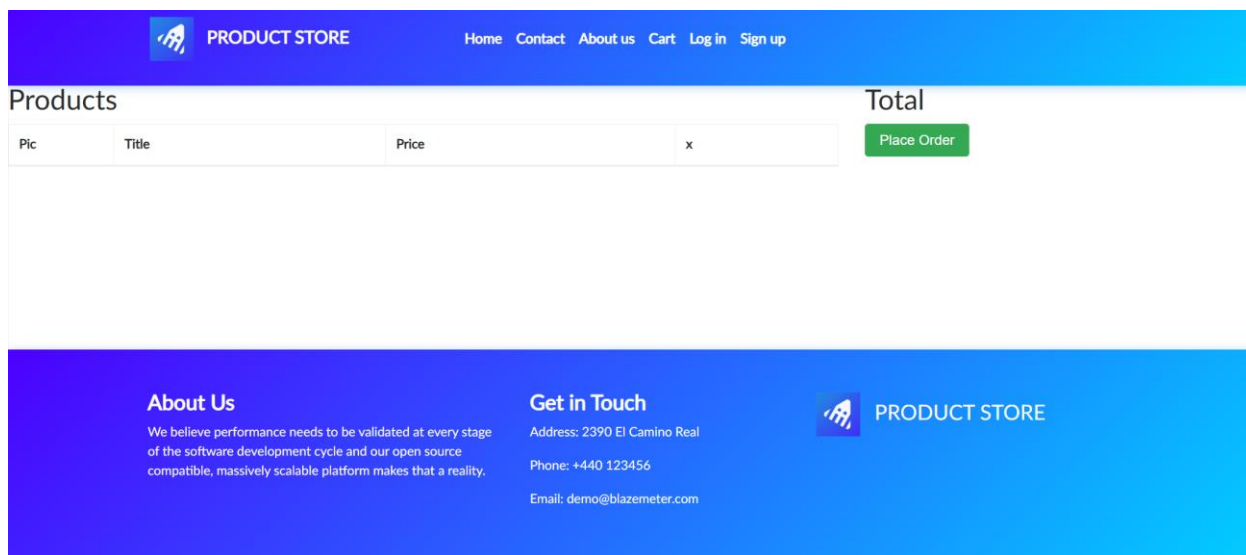


Рисунок 7 - Страница корзины сайта Demoblaze

Тест №4: Проверка добавления товара в корзину

Входные данные:

Категория: Phones

Товар: Samsung Galaxy S6

Действия:

1. Открыть главную страницу сайта Demoblaze.
2. Перейти в категорию Phones.
3. Открыть товар Samsung Galaxy S6.
4. Нажать кнопку Add to cart.
5. Убедиться, что появилось сообщение Product added.
6. Закрыть сообщение Product added.
7. Перейти в раздел Cart.

В результате выполнения теста после нажатия кнопки Add to cart появляется сообщение Product added, подтверждающее добавление товара в корзину. После перехода в раздел Cart в таблице корзины отображается товар Samsung Galaxy S6.

Результаты действий представлены на рисунках (см. рисунки 8 – 11).

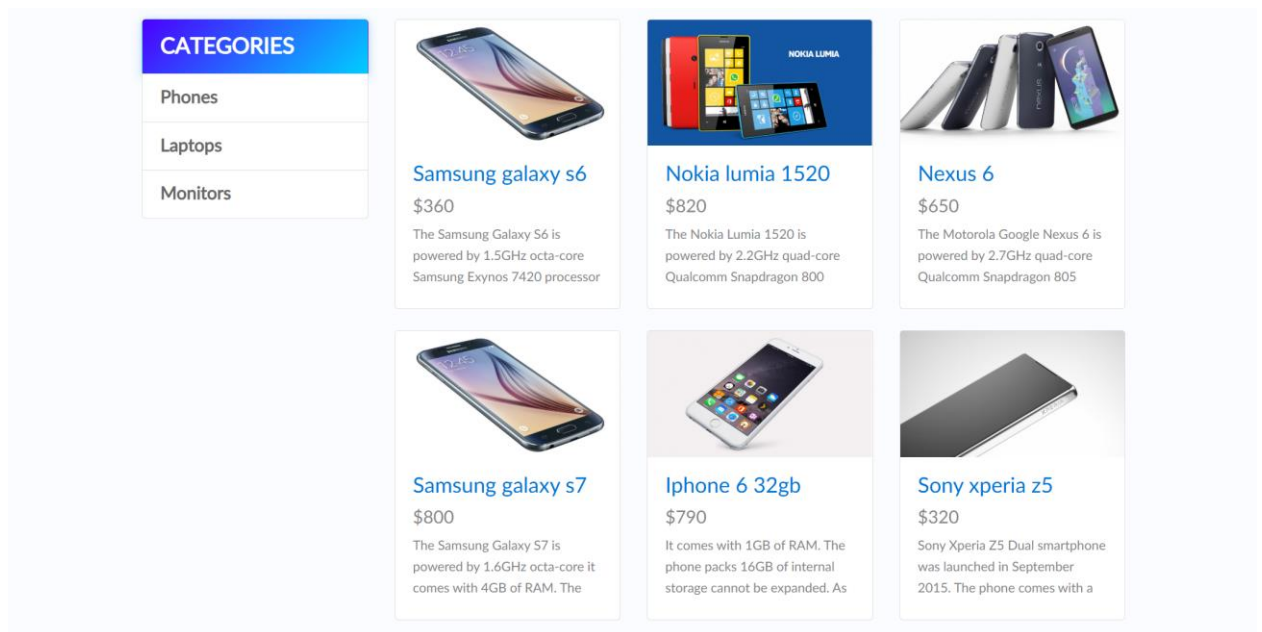


Рисунок 8 - Список товаров категории Phones

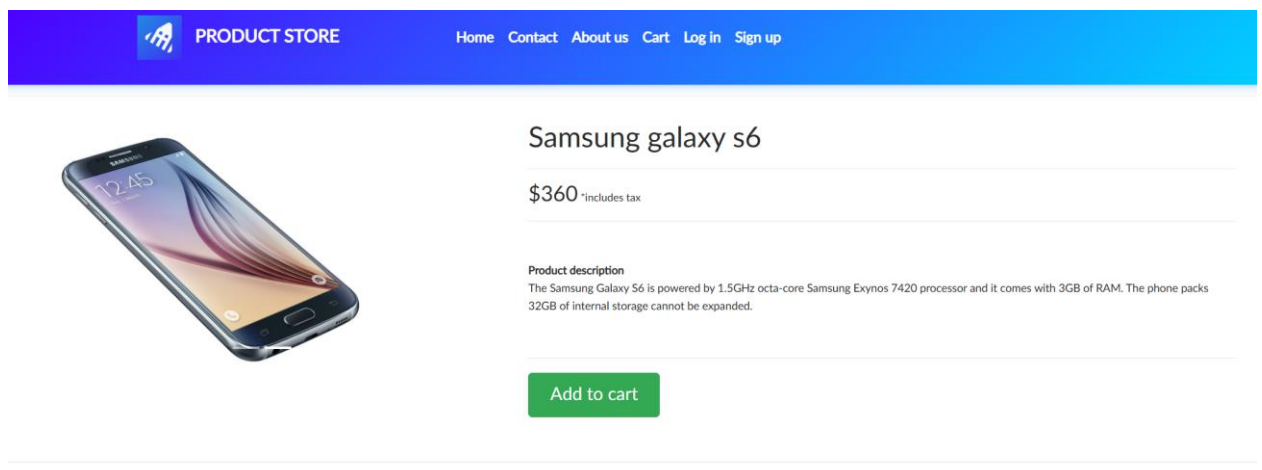


Рисунок 9 - Карточка товара Samsung Galaxy S6

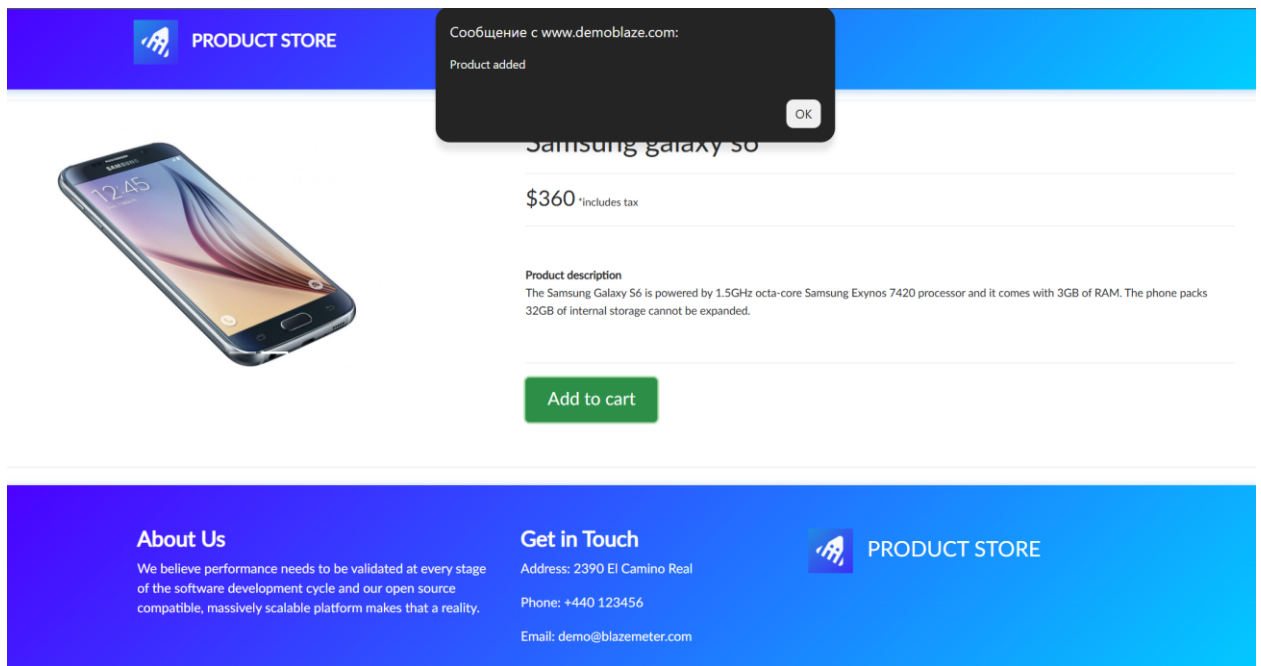


Рисунок 10 - Сообщение Product added после добавления товара

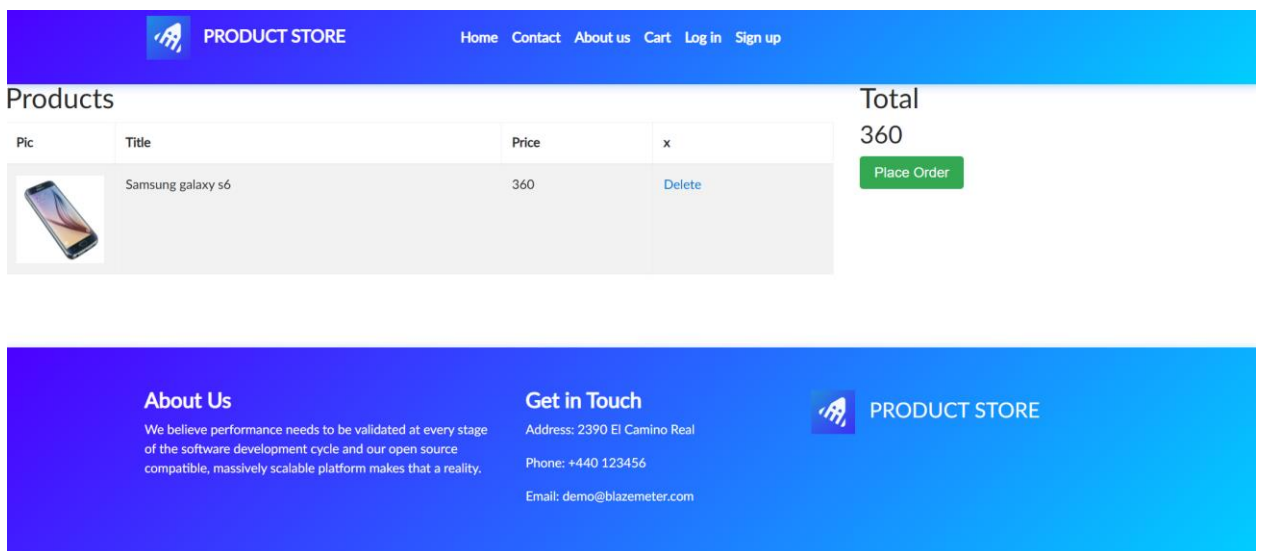


Рисунок 11 - Отображение товара Samsung Galaxy S6 в корзине

Тест №5: Проверка добавления нескольких товаров в корзину

Входные данные:

Категория: Phones

Товары: Samsung Galaxy S6, Nokia Lumia 1520

Действия:

1. Открыть главную страницу сайта Demoblaze.

2. Перейти в категорию Phones.
3. Открыть товар Samsung Galaxy S6.
4. Нажать кнопку Add to cart.
5. Закрыть сообщение Product added.
6. Вернуться на главную страницу через пункт меню Home.
7. Перейти в категорию Phones.
8. Открыть товар Nokia Lumia 1520.
9. Нажать кнопку Add to cart.
10. Закрыть сообщение Product added.
11. Перейти в раздел Cart.

В результате выполнения теста в корзине отображаются оба добавленных товара: Samsung Galaxy S6 и Nokia Lumia 1520.

Результаты действий представлены на рисунках (см. рисунки 12 - 15).

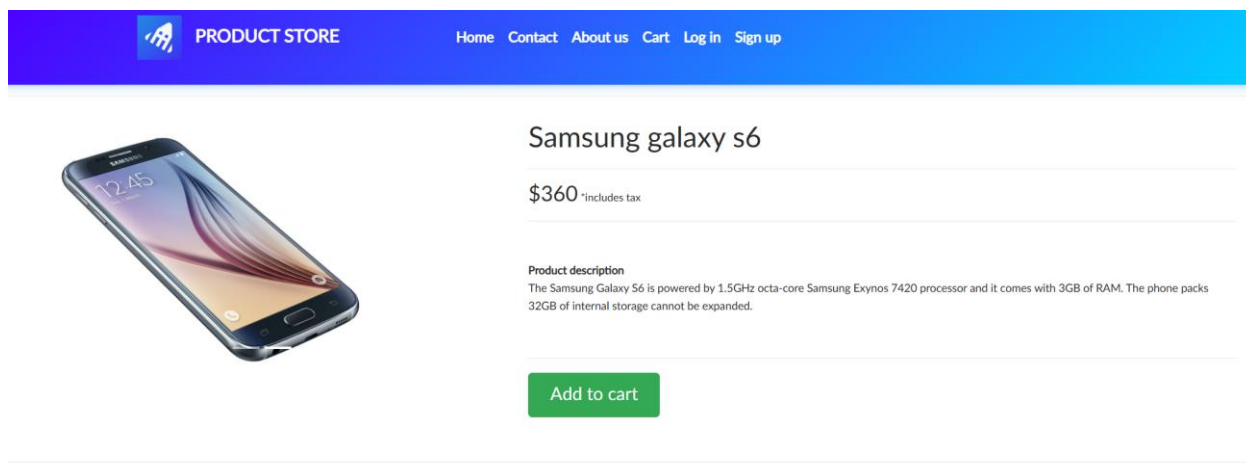


Рисунок 12 - Карточка товара Samsung Galaxy S6

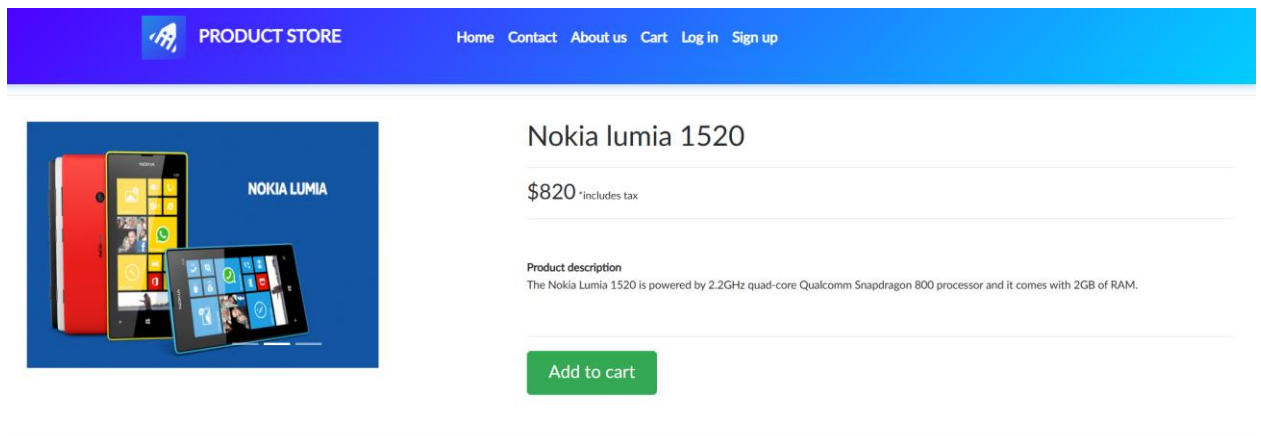


Рисунок 13 - Карточка товара Nokia Lumia 1520

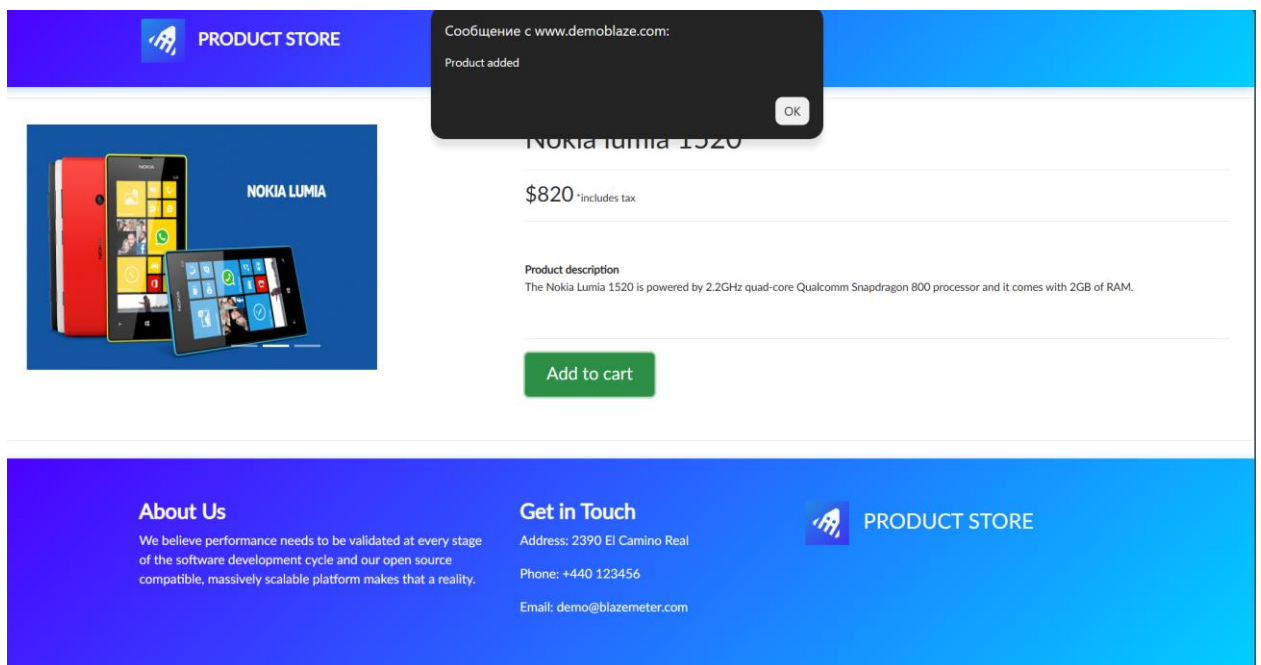


Рисунок 14 - Сообщение Product added после добавления товара

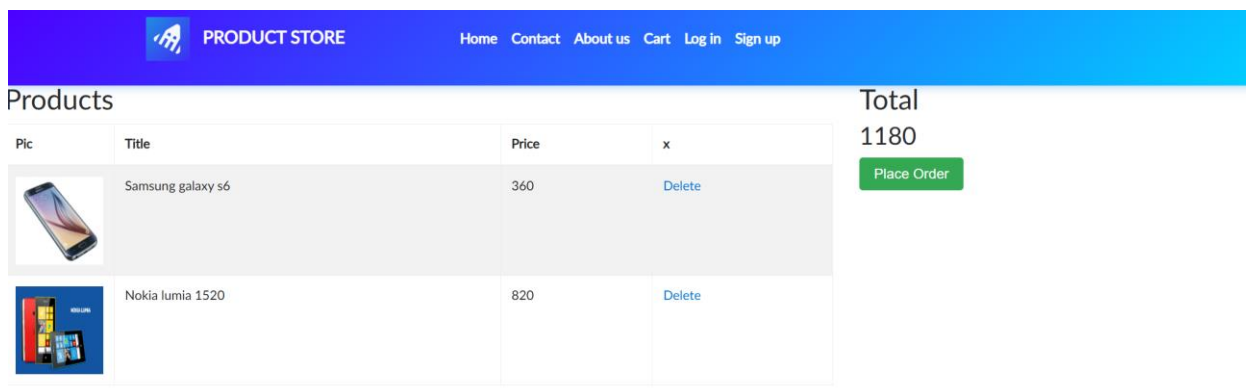


Рисунок 15 - Отображение нескольких товаров в корзине

Тест №6: Проверка удаления товара из корзины

Входные данные:

Категория: Phones

Товар: Samsung Galaxy S6

Действия:

1. Открыть главную страницу сайта Demoblaze.
2. Перейти в категорию Phones.
3. Открыть товар Samsung Galaxy S6.
4. Нажать кнопку Add to cart.
5. Закрывать сообщение Product added.
6. Перейти в раздел Cart.
7. Нажать ссылку Delete напротив товара Samsung Galaxy S6.

В результате выполнения теста товар Samsung Galaxy S6 удаляется из корзины и больше не отображается в списке товаров.

Результаты действий представлены на рисунках (см. рисунки 16, 17).

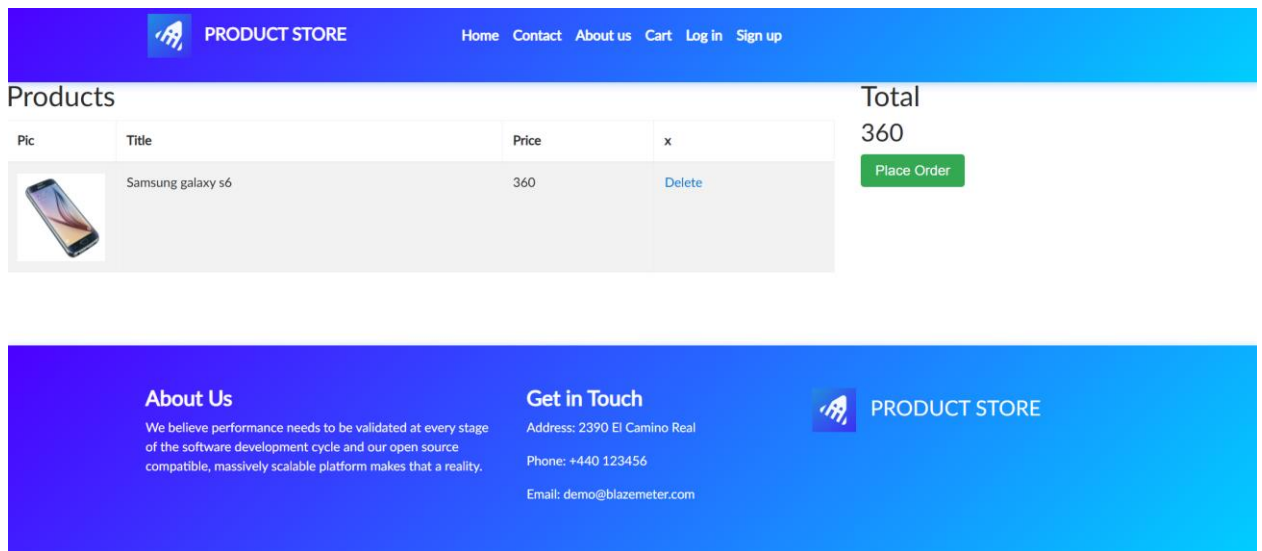


Рисунок 16 - Отображение товара Samsung Galaxy S6 в корзине

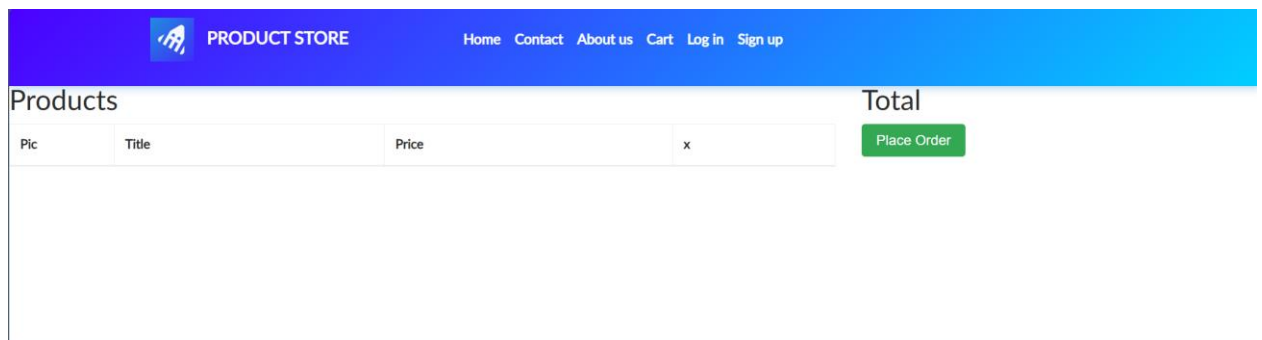


Рисунок 17 - Корзина после удаления товара

Тест №7: Проверка оформления заказа

Входные данные:

Категория: Phones

Товар: Samsung Galaxy S6

Действия:

1. Открыть главную страницу сайта Demoblaze.
2. Перейти в категорию Phones.
3. Открыть товар Samsung Galaxy S6.
4. Нажать кнопку Add to cart.
5. Закрывать сообщение Product added.

6. Перейти в раздел Cart.
7. Нажать кнопку Place Order.
8. Заполнить поле Name значением Test User.
9. Заполнить поле Country значением Russia.
10. Заполнить поле City значением Saint Petersburg.
11. Заполнить поле Credit card значением 1234567890.
12. Заполнить поле Month значением 06.
13. Заполнить поле Year значением 2026.
14. Нажать кнопку Purchase.

В результате выполнения теста открывается форма оформления заказа Place order, пользователь заполняет ее корректными данными и нажимает кнопку Purchase. После успешного оформления заказа отображается сообщение Thank you for your purchase!.

Результаты действий представлены на рисунках (см. рисунки 18 – 21).

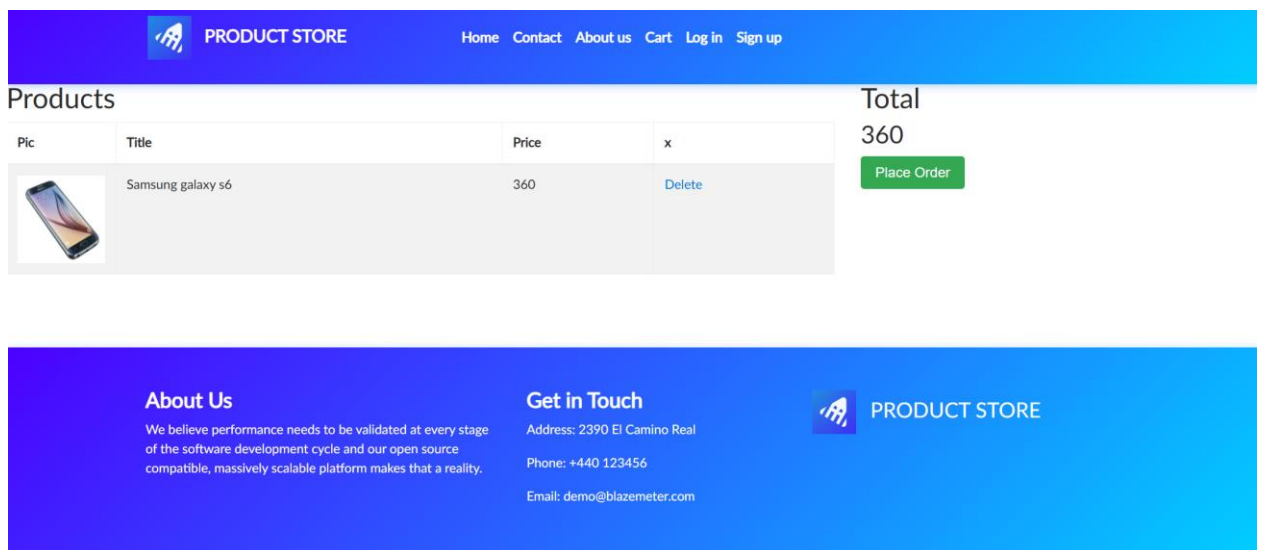


Рисунок 18 - Отображение товара Samsung Galaxy S6 в корзине

PRODUCT STORE

Products

Pic	Title
	Samsung galaxy s6

About Us

We believe performance needs to be validated of the software development cycle and our open compatible, massively scalable platform makes

Total
360
[Place Order](#)

Place order [X]

Total: 360

Name:

Country:

City:

Credit card:

Month:

Year:

[Close](#) [Purchase](#)

PRODUCT STORE

Рисунок 19 - Форма оформления заказа Place order

PRODUCT STORE

Products

Pic	Title
	Samsung galaxy s6

About Us

We believe performance needs to be validated of the software development cycle and our open compatible, massively scalable platform makes

Total
360
[Place Order](#)

Place order [X]

Total: 360

Name:

Country:

City:

Credit card:

Month:

Year:

[Close](#) [Purchase](#)

PRODUCT STORE

Рисунок 20 - Заполненная форма оформления заказа

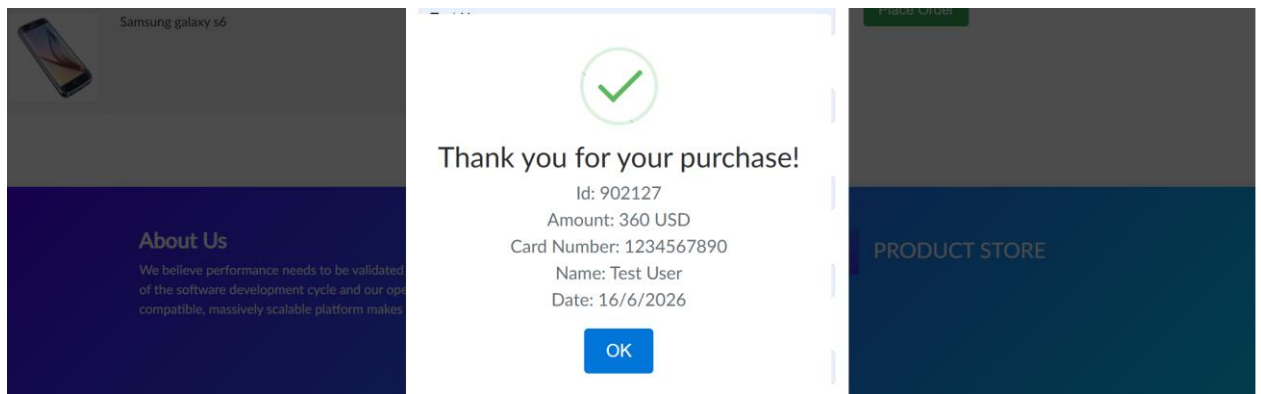


Рисунок 21 - Сообщение об успешном оформлении заказа

Тест №8: Проверка отмены оформления заказа

Входные данные:

Категория: Phones

Товар: Samsung Galaxy S6

Действия:

1. Открыть главную страницу сайта Demoblaze.
2. Перейти в категорию Phones.
3. Открыть товар Samsung Galaxy S6.
4. Нажать кнопку Add to cart.
5. Закрывать сообщение Product added.
6. Перейти в раздел Cart.
7. Нажать кнопку Place Order.
8. Нажать кнопку Close в форме оформления заказа.

В результате выполнения теста форма оформления заказа закрывается без оформления покупки. При этом товар Samsung Galaxy S6 остается в корзине, так как пользователь отменил оформление заказа, но не удалял товар.

Результаты действий представлены на рисунках (см. рисунки 21 – 22).

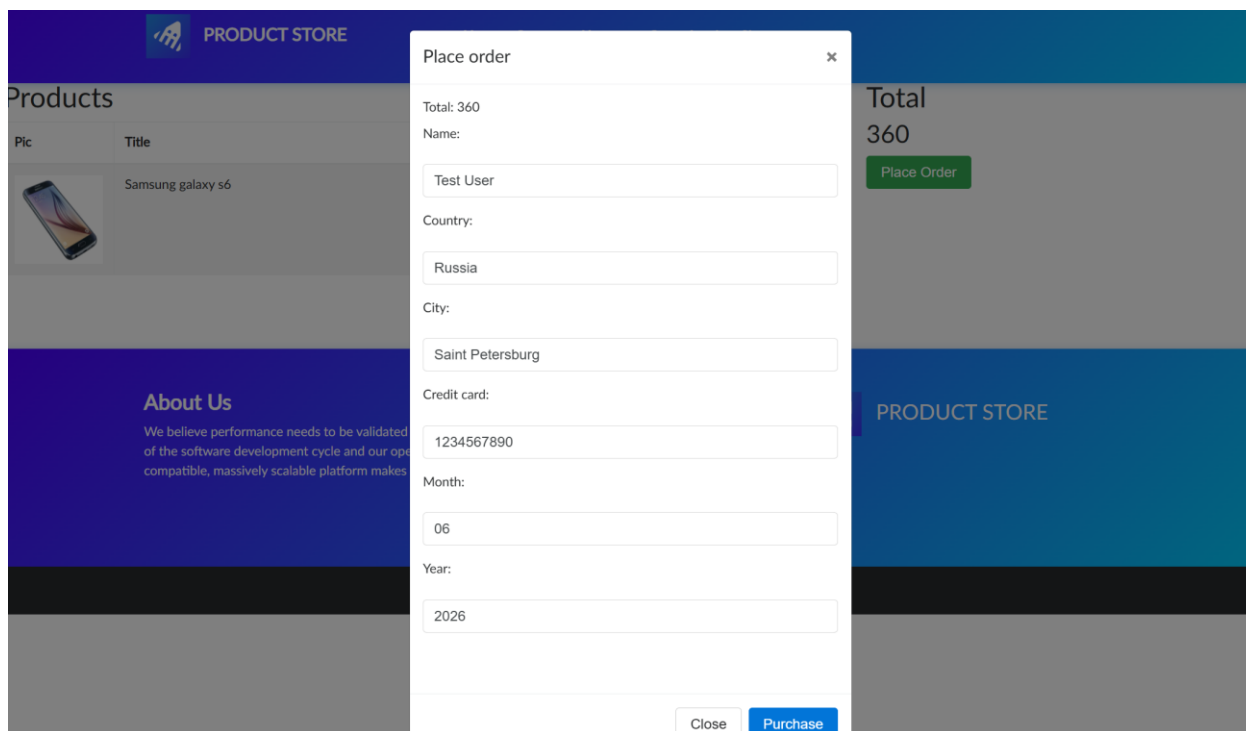


Рисунок 22 - Форма оформления заказа Place order

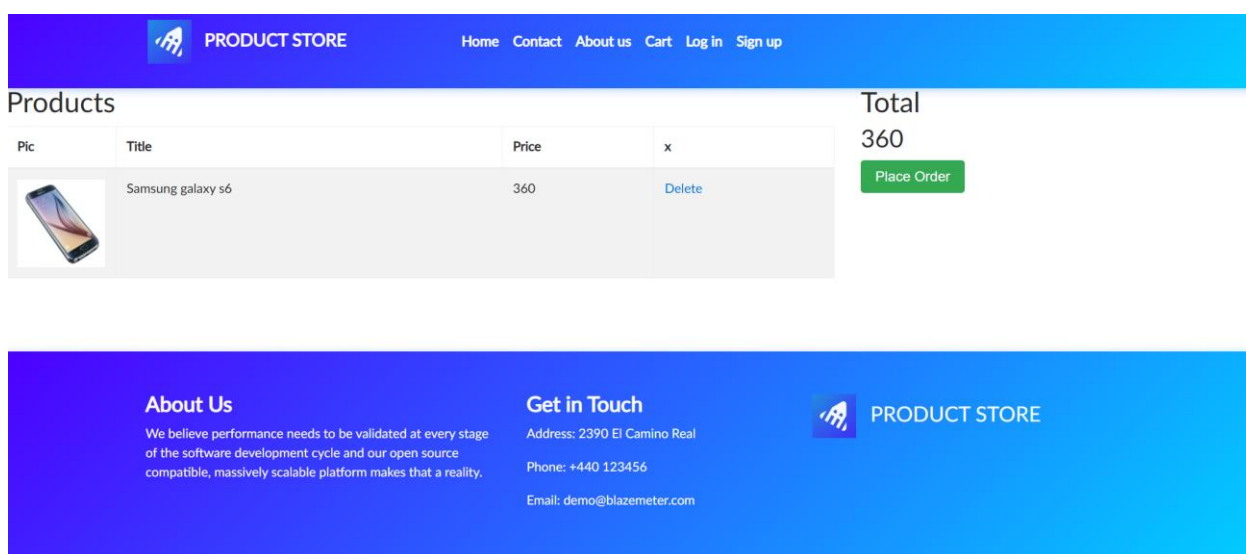


Рисунок 23 - Корзина после отмены оформления заказа

Тест №9: Проверка отображения пустой корзины

Входные данные:

Для проверки пустой корзины входные данные не требуются.

Для добавления товара используется категория Phones и товар Samsung Galaxy S6.

Действия:

1. Открыть главную страницу сайта Demoblaze.
2. Перейти в раздел Cart.
3. Убедиться, что в таблице корзины отсутствуют товары.
4. Вернуться на главную страницу через пункт меню Home.
5. Перейти в категорию Phones.
6. Открыть товар Samsung Galaxy S6.
7. Нажать кнопку Add to cart.
8. Закрыть сообщение Product added.
9. Перейти в раздел Cart.

В результате выполнения теста сначала проверяется, что при переходе в корзину без добавления товаров таблица корзины остается пустой. После добавления товара Samsung Galaxy S6 и повторного перехода в раздел Cart товар отображается в корзине.

Результаты действий представлены на рисунках (см. рисунки 24, 25).

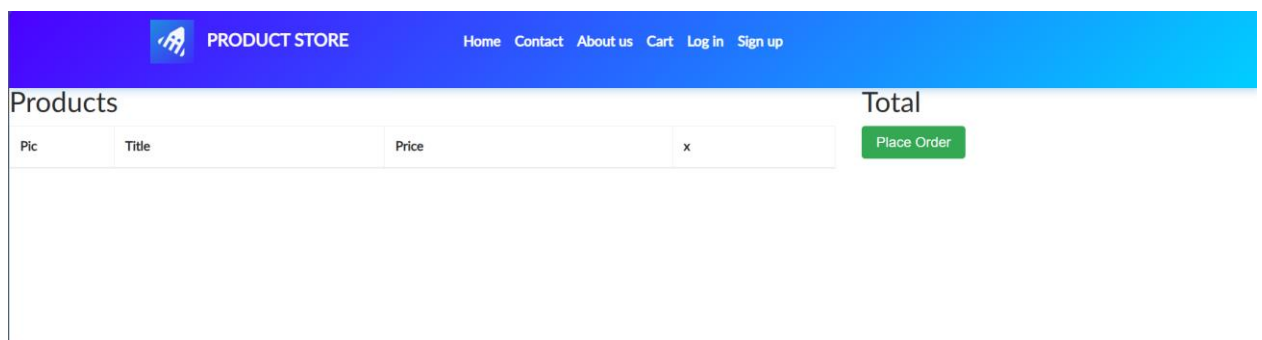


Рисунок 24 - Пустая корзина сайта Demoblaze

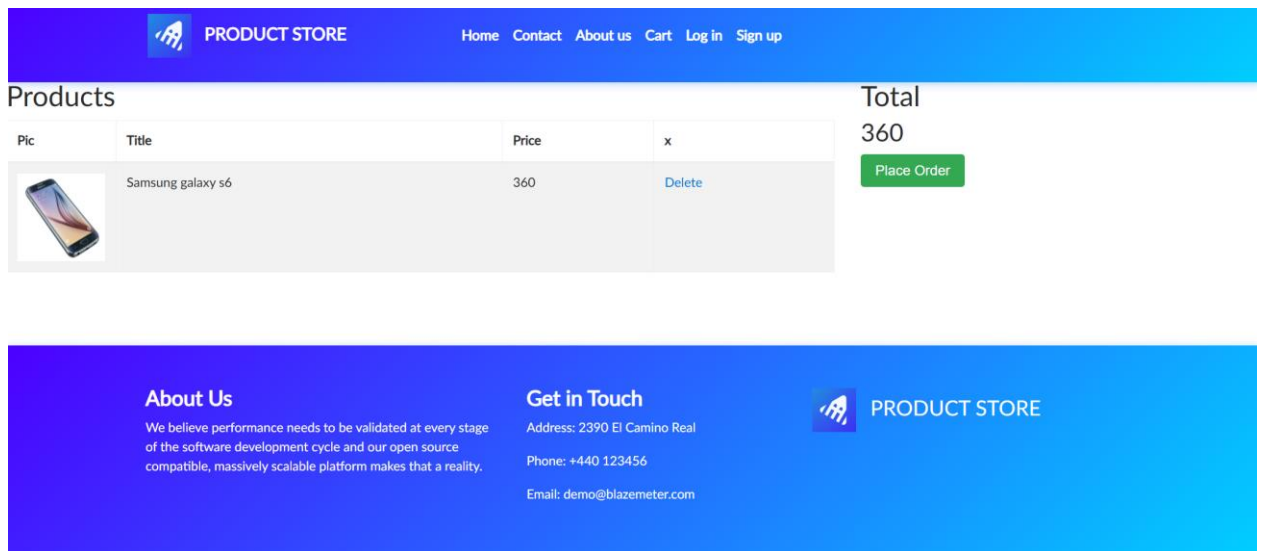


Рисунок 25 - Отображение товара Samsung Galaxy S6 после добавления в корзину.

Результаты выполнения тестов представлены на рис. 26

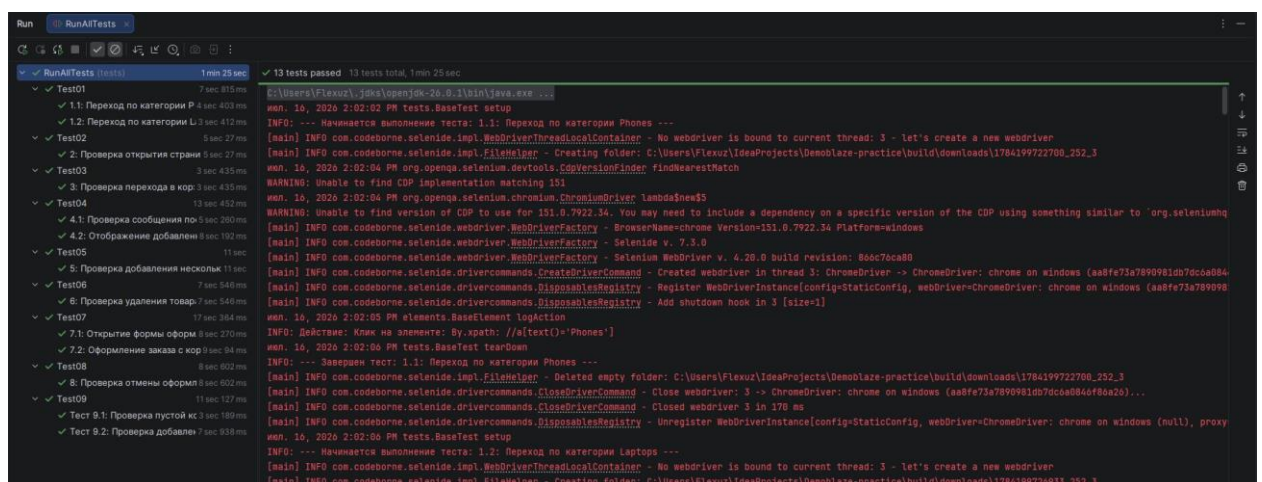


Рисунок 26 - результаты выполнения тестов

2 РЕЗУЛЬТАТЫ РАБОТЫ

В данном разделе приведено описание архитектуры разработанной системы автоматизированного тестирования для сайта Demoblaze. В основу проекта заложен паттерн проектирования Page Object Model (POM), который позволяет разделить логику тестов и описание элементов пользовательского интерфейса, обеспечивая высокую расширяемость и удобство поддержки кода. Проект реализован на языке Java с использованием библиотеки Selenide для взаимодействия с браузером и тестового фреймворка JUnit 5.

Программный комплекс разделен на четыре логических пакета в соответствии с архитектурой POM: элементы страниц (elements), страницы (pages), тесты (tests) и вспомогательные утилиты (utils). Классы тестов не имеют прямого доступа к элементам страниц и взаимодействуют с ними исключительно через методы классов страниц, что соответствует принципам паттерна Page Object Model.

2.1. Описание программных классов и их методов

Программный комплекс разделен на несколько логических пакетов: компоненты, страницы, тесты и вспомогательные утилиты.

Пакет elements (элементы страниц):

BaseElement — базовый класс для всех UI-элементов. Содержит защищенное поле baseElement типа SelenideElement и метод isDisplayed() для проверки видимости элемента на странице. Выступает основой для всех дочерних классов-элементов.

Button — класс для работы с кнопками. Наследуется от BaseElement и расширяет его функциональность методом click(), который выполняет нажатие на кнопку с предварительным логированием действия. Используется для всех интерактивных кнопок на сайте: категории товаров, "Add to cart", "Place Order", "Delete".

Input — класс для работы с полями ввода. Наследуется от BaseElement и предоставляет метод fill(String value) для заполнения поля указанным

значением. Применяется при работе с формами оформления заказа (поля Name, Country, City, Credit Card, Month, Year).

Пакет pages (страницы):

BasePage — базовый класс для всех страниц. Содержит метод waitForElement(String selector), который ожидает появления элемента на странице перед выполнением действий. Обеспечивает стабильность тестов при асинхронной загрузке контента.

MainPage — класс, представляющий главную страницу сайта Demoblaze. Отвечает за навигацию по каталогу товаров. Содержит методы: selectCategory() — выбор категории товаров (Phones, Laptops), openProduct() — открытие карточки товара по названию, goToCart() — переход в корзину, isProductVisible() — проверка наличия товара в списке, clickHome() — возврат на главную страницу, isTitleCorrect() — проверка заголовка страницы.

CartPage — класс, представляющий страницу корзины. Содержит методы для управления товарами: deleteProduct() — удаление товара из корзины, isProductPresent() — проверка наличия товара, isCartEmpty() — проверка пустой корзины. Включает методы оформления заказа: clickPlaceOrder() — открытие модальной формы оформления, fillName(), fillCountry(), fillCity(), fillCard(), fillMonth(), fillYear() — заполнение полей формы, clickPurchase() — подтверждение заказа, closeCheckoutForm() — закрытие формы без оформления. Также реализованы проверочные методы: isCartTableVisible() — проверка отображения таблицы корзины, isPlaceOrderButtonVisible() — проверка наличия кнопки оформления, areHeadersVisible() — проверка заголовков таблицы, isPlaceOrderModalVisible() — проверка открытия модального окна.

ProductPage — класс, представляющий страницу отдельного товара. Содержит методы: clickAddToCart() — добавление товара в корзину, isTitleCorrect() — проверка названия товара, isPriceCorrect() — проверка цены, isDescriptionVisible() — проверка отображения описания, isAddToCartButtonVisible() — проверка наличия кнопки добавления. Для

работы с всплывающими уведомлениями реализованы методы: `acceptAlert()` — подтверждение всплывающего сообщения, `prepareAlertIntercept()` и `getCapturedAlertText()` — для перехвата и проверки текста системных сообщений (например, "Product added").

Пакет tests (тесты):

`BaseTest` — базовый класс для всех тестовых классов. Отвечает за жизненный цикл браузера: перед запуском каждого теста (`@BeforeEach`) настраивает браузер (размер окна 1920x1080, таймаут 10 секунд) и открывает главную страницу Demoblaze. После выполнения каждого теста (`@AfterEach`) закрывает браузер. Использует `LoggerUtil` для логирования начала и завершения тестов.

`Test01` — класс, содержащий тесты навигации по категориям: `testPhonesCategory()` — проверяет переход в категорию Phones и наличие товара Samsung Galaxy S6; `testLaptopsCategory()` — проверяет переход в категорию Laptops и наличие товара Sony Vaio i5.

`Test02` — тест открытия карточки товара `testOpenProductCard()`. Проверяет корректность отображения страницы товара Samsung Galaxy S6: названия, цены (\$360), описания и кнопки "Add to cart".

`Test03` — тест перехода в корзину `testCartNavigation()`. Проверяет, что при открытии корзины отображаются таблица с товарами (колонки Pic, Title, Price) и кнопка "Place Order".

`Test04` — класс, содержащий тесты добавления в корзину: `testAddToCartMessage()` — проверяет появление сообщения "Product added" после добавления товара; `testProductDisplayedInCart()` — проверяет, что добавленный товар отображается в корзине.

`Test05` — тест добавления нескольких товаров `testMultipleProductsInCart()`. Проверяет, что после последовательного добавления Samsung Galaxy S6 и Nokia Lumia 1520 в корзине отображаются оба товара.

Test06 — тест удаления товара из корзины `testDeleteProductFromCart()`. Проверяет, что после нажатия ссылки "Delete" товар Samsung Galaxy S6 исчезает из списка товаров в корзине.

Test07 — класс, содержащий тесты оформления заказа: `testOpenPlaceOrderForm()` — проверяет открытие модальной формы оформления заказа при нажатии кнопки "Place Order"; `testPurchaseWithCorrectData()` — проверяет успешное оформление заказа с заполнением всех полей формы и появление сообщения "Thank you for your purchase!".

Test08 — тест отмены заказа `testClosePlaceOrder()`. Проверяет, что при нажатии кнопки "Close" форма оформления заказа закрывается без выполнения покупки, и товар остается в корзине.

Test09 — класс, содержащий тесты пустой корзины: `testEmptyCart()` — проверяет, что в пустой корзине отсутствуют товары; `testProductEmptyCart()` — проверяет, что после добавления товара в пустую корзину он успешно отображается.

Пакет `utils` (вспомогательные утилиты):

`Constants` — класс-хранилище статических констант, используемых в тестах. Содержит названия товаров (S6, I5, NL), категорий (PH, LAP) и цену (PRICE). Использование констант упрощает поддержку тестов — при изменении тестовых данных достаточно внести правки в одном месте.

`LoggerUtil` — утилитный класс с полем `log` типа `java.util.logging.Logger`. Обеспечивает единообразное логирование в тестах. Используется в `BaseTest` для записи начала и завершения каждого теста, что помогает при отладке и анализе результатов выполнения.

2.2. UML-диаграмма классов

На рисунке (см. рис. 27) приведена UML-диаграмма классов, отражающая архитектуру разработанного тестового фреймворка. Диаграмма наглядно демонстрирует иерархию наследования между классами, а также

взаимосвязи между UI-элементами, страницами и тестовыми классами, что позволяет проследить логику взаимодействия компонентов в рамках паттерна Page Object Model.

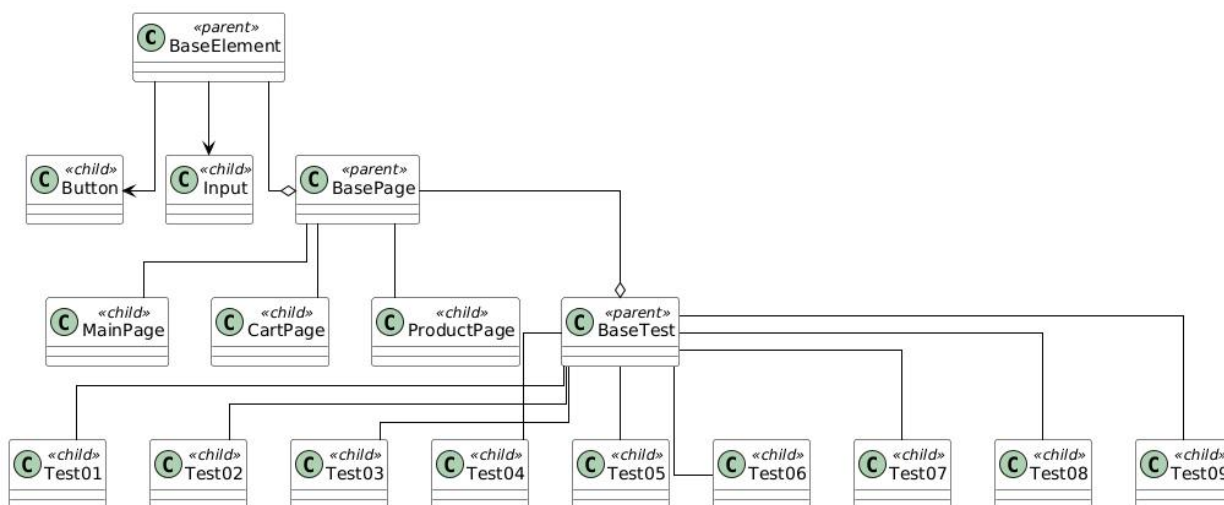


Рисунок 27 - UML-диаграмма

2.3. Реализация взаимодействия компонентов

Взаимодействие между классами построено по иерархическому принципу. Тестовые методы обращаются к методам классов страниц (Page Objects). Страницы, в свою очередь, делегируют управление специфическими блоками интерфейса элементам (Elements).

Например, при вызове метода `clickAddToCart()` в классе `ProductPage` происходит обращение к объекту `Button`, который выполняет нажатие на кнопку. При оформлении заказа методы `fillName()`, `fillCountry()` и другие из класса `CartPage` делегируют ввод данных объектам `Input`.

Такой подход позволяет изменять локаторы элементов в одном месте (в классе элемента) без необходимости править код тестов, что обеспечивает высокую поддерживаемость проекта.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В рамках работы над проектом мною был спроектирован и реализован фундаментальный слой фреймворка, обеспечивающий масштабируемость, стабильность и легкость поддержки автотестов. Детальное описание проделанной работы:

1. Инфраструктура проекта и управление сборкой (pom.xml)

Конфигурация Maven: спроектировал структуру pom.xml, обеспечив корректную компиляцию проекта под Java. Настроил кодировку проекта (UTF-8) для предотвращения проблем с отображением кириллицы в логах и отчетах.

Управление зависимостями: выполнил интеграцию и настройку жизненно важных библиотек:

Selenide: Настройка ядра для взаимодействия с элементами интерфейса.

JUnit 5: Настройка тестового раннера для управления жизненным циклом тестов.

SLF4J: Интеграция логирования для обеспечения прозрачности выполнения тестов.

Стандартизация: обеспечил чистоту конфигурации, исключив лишние зависимости, что позволило ускорить процесс сборки и запуска тестов.

2. Разработка архитектуры UI-элементов (Инкапсуляция)

Иерархия элементов (BaseElement, Button, Input):

Создал абстрактный класс BaseElement, который стал единой точкой входа для любого UI-компонента.

Разработал специализированные классы (Button, Input), которые инкапсулируют логику взаимодействия (например, кастомный метод fill() для полей ввода или click() для кнопок).

3. Архитектура уровней страниц (BasePage)

Создал базовый класс BasePage, который служит каркасом для всех последующих страниц (CartPage, MainPage и др.).

Внедрил общие механизмы ожидания (waitForElement), что повысило стабильность тестов за счет использования динамических ожиданий Selenide.

4. Система логирования и отладки (LoggerUtil)

Реализовал класс LoggerUtil, который стандартизировал вывод информации в консоль.

Настроил интеграцию логирования внутри методов BaseElement, что автоматически дает подробный отчет о каждом действии пользователя (нажатие, ввод текста) с указанием конкретного элемента.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на <ваша_оценка>.

ЗАКЛЮЧЕНИЕ

В ходе выполнения учебной практики были достигнуты поставленные цели и реализованы задачи по созданию системы автоматизированного UI-тестирования демонстрационного интернет-магазина Demoblaze. На первом этапе был проведен анализ основных функциональных возможностей сайта и выделены ключевые пользовательские сценарии, связанные с выбором категории товара, открытием карточки продукта, добавлением товаров в корзину, удалением товаров, оформлением заказа и отменой оформления. На основе проведенного анализа был подготовлен чек-лист, который охватывает основной путь пользователя от выбора товара до завершения покупки.

Вторая задача, связанная с разработкой архитектуры проекта, была решена с применением объектно-ориентированного подхода на языке Java и паттерна Page Object Model. В проекте были реализованы классы страниц MainPage, ProductPage и CartPage, которые отвечают за работу с главной страницей, карточкой товара и корзиной. Такой подход позволил отделить тестовые сценарии от логики взаимодействия с интерфейсом сайта и сделать структуру проекта более понятной и удобной для дальнейшего сопровождения.

Также в проекте были созданы базовые классы для работы со страницами и элементами интерфейса: BasePage, BaseElement, Button и Input. Они позволяют использовать общие методы для ожидания элементов, кликов по кнопкам, заполнения полей и проверки отображения элементов. Это помогло избежать дублирования кода и привести взаимодействие с элементами сайта к единому виду. Основные тестовые данные, такие как названия категорий, товаров и цена, были вынесены в класс Constants, что упростило поддержку тестов.

Третья задача, включающая программную реализацию автотестов и подготовку документации, была выполнена с использованием Selenide и JUnit 5. В проекте были реализованы тесты, проверяющие переход по категориям Phones и Laptops, открытие карточки товара Samsung Galaxy S6, переход в корзину, добавление одного и нескольких товаров, удаление товара, открытие

формы заказа, оформление покупки с корректными данными, отмену оформления заказа и проверку пустой корзины. Базовый класс `BaseTest` обеспечивает запуск каждого теста с открытием сайта `Demoblaze`, настройкой размера окна браузера, таймаута ожидания и закрытием браузера после выполнения проверки.

Для отслеживания выполнения тестов в проекте было реализовано базовое логирование с помощью класса `LoggerUtil`. Логирование используется при запуске и завершении тестов, а также при выполнении действий с элементами интерфейса. В отличие от исходного шаблона, в данном проекте не использовалась система `Log4j2`, так как логирование выполнено средствами стандартного Java-логгера.

В целом, работа над проектом позволила закрепить навыки проектирования структуры автоматизированных UI-тестов, применения паттерна `Page Object Model` и работы с инструментами `Selenide` и `JUnit 5`. Разработанная система автотестов проверяет основные функции интернет-магазина `Demoblaze` и может быть использована для повторной проверки стабильности работы пользовательских сценариев. Итоговые материалы проекта, включая исходный код, чек-лист, UML-диаграммы и отчетную документацию, были подготовлены для размещения в `GitHub`-репозитории и дальнейшего использования в рамках учебной практики.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Selenium Documentation. Page object models [Электронный ресурс]. URL: https://www.selenium.dev/documentation/test_practices/encouraged/page_object_models/ (дата обращения: 15.07.2026).
2. Selenide. Official documentation [Электронный ресурс]. URL: <https://selenide.org/documentation.html> (дата обращения: 15.07.2026).
3. JUnit 5 User Guide [Электронный ресурс]. URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 07.07.2026).
4. <https://github.com/fl3xuz/Demoblaze-practice>
5. Apache Maven. What is Maven? [Электронный ресурс]. URL: <https://maven.apache.org/what-is-maven.html> (дата обращения: 14.07.2024).
6. Oracle Java Documentation. Package java.util.logging. — URL: <https://docs.oracle.com/en/java/javase/21/docs/api/java.logging/java/util/logging/package-summary.html> (дата обращения: 14.07.2026).